

Node.js Modules Guide (CommonJS & ES Modules)

Modules in Node.js help split code into multiple files. You can export code from one file and import it into another. Node.js mainly supports:

1. CommonJS (Traditional Node.js)
2. ES Modules / Modern JavaScript

1. CommonJS (Traditional Node.js)

CommonJS uses:

- `require()`
- `module.exports`

```
commonjs-demo/
├──
├── math.js
└── app.js

// math.js

function add(a, b) {
  return a + b;
}

function sub(a, b) {
  return a - b;
}

module.exports = {
  add,
  sub
};

// app.js

const math = require("./math");

console.log(math.add(10, 5));
console.log(math.sub(10, 5));
```

Run using:
`node app.js`

2. ES Modules (Modern JavaScript)

ES Modules use:

- `import`
- `export`

Important: Add this in `package.json`:

```
{
  "type": "module"
}
```

```
esm-demo/
├──
├── math.js
├── app.js
└── package.json

// math.js
export function add(a, b) {
```

```

    return a + b;
}

export function sub(a, b) {
    return a - b;
}

// app.js

import { add, sub } from "./math.js";

console.log(add(20, 10));
console.log(sub(20, 10));

```

3. Default Export

```

// math.js

export default function multiply(a, b) {
    return a * b;
}

// app.js

import multiply from "./math.js";

console.log(multiply(5, 4));

```

4. CommonJS vs ES Modules

Feature	CommonJS	ES Modules
Import	require()	import
Export	module.exports	export
Default in Node	Yes	Needs "type":"module"
Type	Old	Modern
Execution	Synchronous	Asynchronous

5. Important Syntax Quick Revision

CommonJS Export:

```
module.exports = data;
```

CommonJS Import:

```
const data = require("./file");
```

ES Module Export:

```
export default data;
```

OR

```
export { add, sub };
```

ES Module Import:

```
import data from "./file.js";
```

OR

```
import { add, sub } from "./file.js";
```

6. Which One Should You Learn?

- Old Node.js projects mostly use CommonJS.
- React, Vite, and modern applications use ES Modules.
- Learning both is important because companies use both styles.